

A Additional Experimental Details

A.1 Implementation Details

We implement DIGEST in Python 3.11.8 with PyTorch 2.2.2 and CUDA 12.1. The Transformer denoiser consists of four Transformer blocks with hidden dimensions [32, 32, 24, 24]. Sparse message passing is activated from layer $\ell_{\text{sp}} = 1$, with the row-wise and column-wise budgets both set to $k_{\text{sp}} = 8$.

We train the denoiser using Adam with a learning rate of 10^{-3} , weight decay of 5×10^{-4} , batch size of 128, and dropout rate of 0.1. The forward noising process uses $T = 1000$ steps during training. During inference, we employ DDIM-based sampling to reduce the reverse process to 10 steps. By default, the reverse sampling procedure generates $K = 1$ candidate matching from the denoiser, and the weight of the base matrix is set to $\lambda = 1.0$.

For beam search, the default fraction of anchored node pairs is 0. We use beam width $B = 8$, successor branch factor $W = 32$, truncation factor $Q = 8$, and search depth $r = |V_1|$, so every node of the first graph is explored. All approaches are evaluated on a single server equipped with AMD EPYC 7542 CPUs, 64 GB of memory, and one NVIDIA RTX 4090 GPU with 24 GB of memory.

A.2 Constrained Greedy Decoding

Given a calibrated score matrix $\mathbf{P} \in \mathbb{R}^{n_1 \times n_2}$, the decoder constructs a binary matching while enforcing that every row and column is selected at most once. As illustrated below, it proceeds in three stages. First, it selects the highest-scoring column in every row, producing a row-wise top-1 candidate matrix. Second, when several rows select the same column, it retains only the candidate with the largest score in that column; exact-score ties are broken deterministically by row index. The surviving entries form a legal partial matching because no two selected pairs share a row or column. These two parallel stages are applied at every reverse step in the accelerated implementation. At the final reverse step, let U_1 and U_2 denote the unmatched row and column sets, respectively. The decoder repairs the residual region by selecting the global maximum among the currently feasible scores, adding the corresponding pair, and masking its row and column. It performs at most $R = 6$ such global-greedy rounds, stopping earlier if either residual set is exhausted. If both sets remain nonempty after this budget, the decoder enumerates their indices in ascending order and, in one batched operation, pairs the first $m = \min(|U_1|, |U_2|)$ corresponding positions. This deterministic fallback preserves the one-to-one constraint and guarantees termination without any further score-matrix scans. The result is a legal injective matching that saturates the smaller graph.

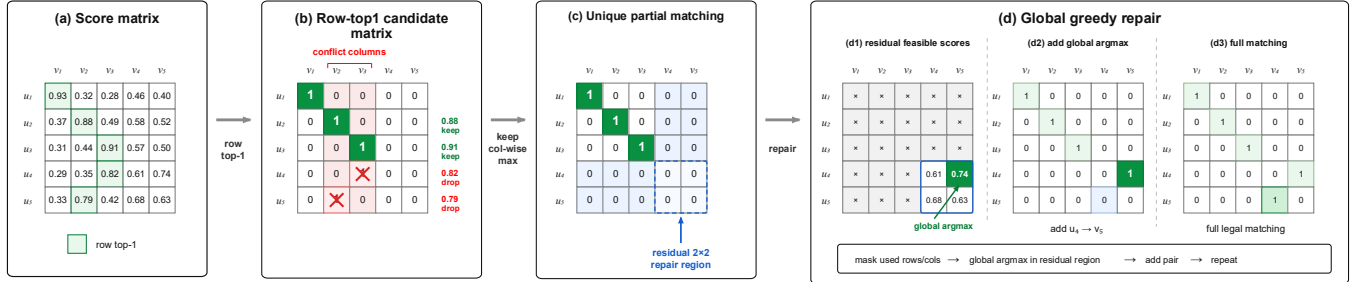


Figure 1: Constrained greedy decoding. Row-wise top-1 selection first proposes one candidate per source node. Column conflicts are resolved by retaining only the highest-scoring candidate in each conflicted column. The decoder then performs at most six global-greedy repair rounds in the residual feasible region; a deterministic batched fallback pairs corresponding residual nodes until the smaller side is exhausted.

Time and Space Complexity. The row-wise top-1 reduction scans the score matrix once and therefore requires $O(n_1 n_2)$ work. Conflict resolution processes the n_1 row candidates and aggregates them by column, requiring $O(n_1 + n_2)$ additional work. Let $q = \min(|U_1|, |U_2|)$ be the number of pairs required to saturate the smaller residual side after conflict resolution at the final reverse step. Each global-greedy repair round performs a masked argmax over all $n_1 n_2$ score entries and then masks one row and one column. With the fixed budget $R = 6$, this phase performs only $\min(q, R)$ sequential rounds and costs $O(\min(q, R) n_1 n_2)$. If $q > R$, constructing the remaining index lists and pairing their first $q - R$ positions costs $O(n_1 + n_2)$. A final-step full decoding operation therefore costs

$$O((1 + \min(q, R)) n_1 n_2 + n_1 + n_2) = O(n_1 n_2),$$

because R is a constant. Across T_{rev} reverse steps, row-wise selection and conflict resolution run at every step, whereas bounded residual repair runs only at the final step. The decoder work per sample is thus $O((T_{\text{rev}} + R) n_1 n_2)$; for the fixed inference schedule $T_{\text{rev}} = 10$ and $R = 6$, this is $O(n_1 n_2)$. For K independently generated samples, the aggregate decoder work is $O(K(T_{\text{rev}} + R) n_1 n_2)$. These bounds cover only constrained decoding and exclude denoising as well as intermediate re-noising or partial-resampling operations. The score, mask, and output matrices require $O(n_1 n_2)$ memory per sample. For a GPU batch of b padded graph pairs, aggregate work and memory scale by b , while the score-based repair has at most $R = 6$ data-dependent sequential rounds across the batch. GPU batching parallelizes the matrix reductions and graph pairs but does not change these work bounds.

A.3 Dataset Statistics

The following table summarizes the number of graph pairs, node-label cardinality, and graph-size ranges of the four benchmarks used in our experiments.

Table 1: Statistics of the graph datasets. Here, #Pairs is the total number of constructed graph pairs, $|L|$ is the node-label cardinality, and $|V|$ and $|E|$ denote the numbers of nodes and edges, respectively.

Dataset	#Pairs	$ L $	$ V $			$ E $		
			min	max	avg	min	max	avg
IMDB	330,891	1	7	89	13.00	12	1467	65.94
OGB-code2	150,000	97	50	100	71.28	49	99	70.28
OGB-molhiv	150,000	314	20	50	27.81	18	63	30.12
OGB-molpcba	150,000	150	20	50	27.50	17	61	29.84